



Introduction To AI: Project Report

The "Algiers in 24 Hours" Tourist Optimizer

**MOHAMMED AL GHAZALI
MEGHABBER, G10**

*mohammed-al-
ghazali.meghabber@ensia.edu.dz*

**MOHAMED RIAD DOUKHA,
G8**

*mohamed-
riad.doukha@ensia.edu.dz*

**MOHAMED AYOUB
BENAMROUCHE, G12**

*mohamed-
ayoub.benamrouche@ensia.edu.dz*

MEHDI BENHAMOUD, G9
mehdi.benhamoud@ensia.edu.dz

RAYANE MAAROUF, G9
rayane.maarouf@ensia.edu.dz

AYOUB OUAZENE, G5
ayoub.ouazene@ensia.edu.dz

Contents

1	Abstract	3
2	Introduction	3
3	State of the Art	4
3.1	Exact Methods and Mathematical Programming (1980s–1990s)	4
3.2	Metaheuristics and Local Search Algorithms (2000s–2010s)	4
3.3	Modern Approaches, mainly AI-based (Machine Learning, Metaheuristics)	4
3.4	Concerning Our Project	4
4	Dataset Building	5
4.1	Data Collection Phase	5
4.2	Data Cleaning and Standardization	5
4.3	Dataset Schema and Characteristics	5
4.4	Travel Time Matrix Generation	5
4.5	Summary Statistics and Exploratory Data Analysis	6
5	Problem Solving Techniques	6
5.1	Problem Formulation	6
5.2	Objective Functions and Heuristics	7
5.2.1	Unified Evaluation (Fitness) Function	7
5.2.2	Algorithm-Specific Heuristics	8
6	Application Design	8
6.1	Frontend and UI/UX Design Approach	9
6.2	Backend Architecture	9
6.3	Database Design	10
7	Results and Analysis	11
7.1	Evaluation Setup	11
7.2	Results	11
7.3	Deep Dive — Main Algorithms	11
7.3.1	Greedy Best-First Search	11
7.3.2	Simulated Annealing	11
7.3.3	Genetic Algorithm	11
7.4	Additional Algorithms	12
7.4.1	Strong Performers	12
7.4.2	Weaker Performers	12
7.5	Time Budget and Runtime Analysis	12
7.6	Key Insights	13
8	Discussion	13
8.1	Results Interpretation	13
8.2	Limitations	13
8.3	Potential improvements	14
8.4	Future Work	14
9	Conclusion	14
10	References	15

11 Appendix A	16
12 Appendix B: Team Contributions	18

1 Abstract

This report presents the development of an intelligent system for tourist itinerary optimization in Algiers, leveraging historical and cultural landmark data alongside user-defined preferences. The primary objective is to maximize cultural and historical value within a user-specified time budget, while respecting site-specific constraints such as opening hours and visit durations. We model the Tourist Trip Design Problem (TTDP) using a multi-paradigm approach encompassing search algorithms (Greedy Best-First and Informed Search), metaheuristics (Simulated Annealing, Hill Climbing, and Genetic Algorithms), swarm intelligence (Ant Colony System and Artificial Bee Colony), hybrid methods (ACS with Simulated Annealing), and Constraint Satisfaction Problems (CSP) for validation. The system integrates external geospatial data sources, applies AI optimization techniques for itinerary exploration, and delivers personalized travel recommendations through a user-friendly web application. A comparative evaluation highlights the trade-offs between different strategies in terms of solution quality, computational efficiency, and practical applicability in real-world urban tourism scenarios.

Keywords: Tourist Trip Design Problem, Algiers Itinerary Optimization, Search Algorithms, Metaheuristics, Swarm Intelligence, Constraint Satisfaction Problems, AI Optimization

2 Introduction

In today's fast-paced tourism landscape, particularly in culturally rich cities like Algiers, travelers face the daunting challenge of designing optimal itineraries that maximize historical and cultural value while adhering to personal time constraints, site-specific requirements (such as opening hours and visit durations), and logistical realities of urban navigation. The Tourist Trip Design Problem (TTDP) encapsulates this complexity, where traditional approaches often lead to suboptimal experiences—either rushed visits that miss key insights or inefficient routes that waste precious time. Addressing this problem is crucial as it empowers tourists to make informed decisions, enhances their overall satisfaction, and promotes sustainable tourism by optimizing resource utilization in heritage sites.

To tackle this, our project implements a comprehensive suite of intelligent optimization algorithms, each offering distinct strategies for itinerary construction under user-defined constraints. These include search-based methods, metaheuristics, swarm intelligence techniques, and constraint validation approaches, enabling personalized and efficient trip planning.

- **Greedy Best-First Search:** Constructs itineraries by iteratively selecting the highest-value landmarks available, while ensuring time budgets and site constraints are respected for rapid, near-optimal solutions.
- **Simulated Annealing:** Introduces controlled randomness to escape local optima, accepting suboptimal itineraries initially and gradually refining choices through temperature-based cooling, allowing for robust handling of complex constraints.
- **Hill Climbing:** Iteratively improves itineraries by making incremental adjustments toward higher-value configurations, suitable for quick refinements within constrained search spaces.
- **Genetic Algorithms:** Evolves populations of potential itineraries through selection, crossover, and mutation, mimicking natural evolution to discover diverse, high-quality solutions over generations.
- **Ant Colony System (ACS):** Simulates ant foraging behavior to build pheromone trails that guide route optimization, emphasizing collective intelligence for dynamic pathfinding in landmark networks.
- **Artificial Bee Colony (ABC):** Employs bee-inspired foraging strategies, dividing the search into employed, onlooker, and scout bees to explore and exploit itinerary options efficiently.
- **Hybrid ACS with Simulated Annealing:** Combines ACS's global exploration with Simulated Annealing's local refinement, creating a synergistic approach for superior convergence and constraint satisfaction.

- **Constraint Satisfaction Problems (CSP):** Validates itinerary feasibility by ensuring all temporal, spatial, and site-specific constraints are met, serving as a complementary verification layer for generated solutions.

This multi-paradigm framework not only addresses the TTDP's intricacies but also provides tourists with flexible, AI-driven tools to create memorable, well-balanced experiences in Algiers, fostering cultural appreciation and efficient travel planning.

3 State of the Art

The Tourist Trip Design Problem (TTDP), also known as the Orienteering Problem with Time Windows (OPTW) or Team Orienteering Problem (TOP) in tourism, has received a lot of attention from researchers in operations research and artificial intelligence. They have widely studied combinatorial optimization methods to solve itinerary planning, especially for cultural sites in cities. Important works include :

3.1 Exact Methods and Mathematical Programming (1980s–1990s)

Early research on the Orienteering Problem with Time Windows (OPTW) used mathematical programming, where computers solve the problem step-by-step to find the absolute best route. These exact methods guarantee that the solution is truly optimal, they consider all the possible routes and constraints mathematically.

However, these approaches have a major weakness: they become too slow as the problem gets bigger. OPTW is an NP-hard problem, meaning that adding just a few more locations can make the computation time explode. Also, these methods assume that travel times are fixed and don't change, which is not realistic in real traffic situations where delays happen.

3.2 Metaheuristics and Local Search Algorithms (2000s–2010s)

To solve this speed problem, researchers turned to simpler methods called metaheuristics—like Genetic Algorithms, Tabu Search, and Ant Colony Optimization. These methods work differently: instead of finding the perfect solution, they find good solutions quickly by making smart choices about which locations to visit and in what order.

The problem is that these methods need a lot of manual tuning to work well. They often get stuck in local solutions and can't improve further. Also, they plan the entire route before the trip starts, so they can't adapt if something changes during the journey, like new delivery requests or unexpected traffic.

3.3 Modern Approaches, mainly AI-based (Machine Learning, Metaheuristics)

More recently, researchers have explored AI-based methods to handle the complexity of OPTW. Techniques such as Machine Learning (ML) and Reinforcement Learning (RL) have been used to go beyond the simple mathematical models, offering flexibility to handle complex constraints and unpredictable travel times. As of early 2026, the latest approach is "Hybrid Neural-Decoupled" solvers, such as the DeCoST framework. This method works in two stages: a neural network predicts the best route, while a mathematical solver optimizes the arrival times. This allows the system to handle changing profits and complex time windows, and it runs 6.6 times faster than traditional methods.

However, these AI methods have their own weaknesses. The biggest problem is that they act like a "black box" "you don't always understand why they make certain decisions. They also need huge amounts of good-quality training data. While they are extremely fast once trained, they can fail when facing situations that were not in their training data, such as extreme weather or unusual events.

3.4 Concerning Our Project

Our approach is special because it uses smart methods like metaheuristics, such as Simulated Annealing, Genetic Algorithms, Ant Colony System, and Artificial Bee Colony, to plan trips that fit each person's needs. Unlike

old methods that are fixed and can't change, these techniques adapt easily to different time limits, visit times, and favorite places. They also work well with hybrid combinations, like mixing Ant Colony with Simulated Annealing, to find better routes quickly and handle surprises like closed landmarks or new requests during the trip. This makes the system personal and ready for real-world changes.

4 Dataset Building

4.1 Data Collection Phase

- **Landmarks Data:** We initialized the collection using web scraping techniques. By leveraging **Playwright** and **BeautifulSoup**, we extracted an initial raw list of over 700 potential attractions from *Trip.com*. To enrich this data, we queried the **Wikipedia API** to fetch descriptions and categorical types. Geographical coordinates (Latitude and Longitude) were obtained using **Overpass Turbo** (based on OpenStreetMap). We manually added precise visit durations and opening hours via Google Maps to satisfy algorithmic constraints, assigned interest scores (out of 10) based on public feedback, and sourced images from platforms like Pinterest.
- **Hotels Data:** The base dataset for Algiers accommodations was extracted using **Outscraper**, utilizing strict search parameters and hashtags to collect all viable hotels and their geographical coordinates within the city limits.

4.2 Data Cleaning and Standardization

- **Filtering:** The initial 700+ landmarks were heavily filtered to isolate the **67 most interesting and relevant landmarks**, eliminating redundancies and low-value POIs.
- **Standardization:** We normalized the landmark categories, mapping them into **11 unique types** (e.g., Museum, Historical Site, Park) to facilitate user filtering.
- **Translation:** All text attributes, including names and descriptions, were standardized into English
- **Validation:** We manually cross-referenced all GPS coordinates with Google Maps for spatial accuracy and verified the operational existence of each extracted hotel.

4.3 Dataset Schema and Characteristics

Table 1: Landmarks Dataset Schema

Attribute	Description	Attribute	Description
Name	Standardized name .	Coordinates	Geo-location (Longitude, Latitude).
Description	Short textual summary about the Landmark.	Visit Duration	Est. time (in mins) required for visit.
Type	Categorized classification of POI.	Opening Hours	24-element boolean array per day (1=open, 0=closed for each hour) to enforce temporal constraints.
Rating	Interest score (out of 10) for weight.	Image URL	Direct link to site's visual representation.

The **Hotels Dataset** consists of **180+ validated accommodations**, containing simply the Name and GPS Coordinates, serving strictly as geographical origin nodes.

4.4 Travel Time Matrix Generation

To ensure realistic travel constraints, we generated a comprehensive **Time Matrix** using the **OSRM (Open Source Routing Machine)** API. By querying collected GPS coordinates, we obtained real-world estimated travel times between all point pairs (Landmark-to-Landmark and Hotel-to-Landmark), serving as the fundamental cost function for our optimization solvers.

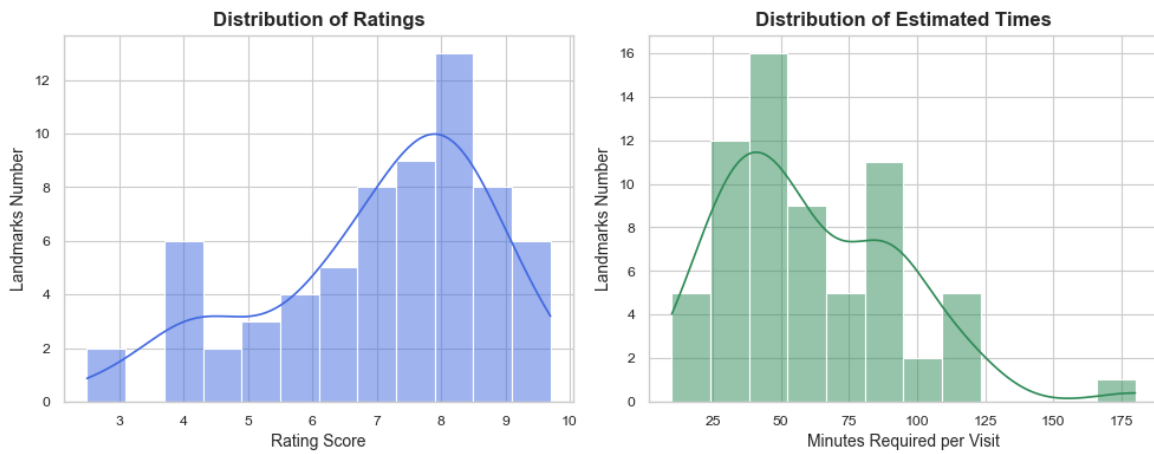
4.5 Summary Statistics and Exploratory Data Analysis

Table 2: Landmarks Category Distribution

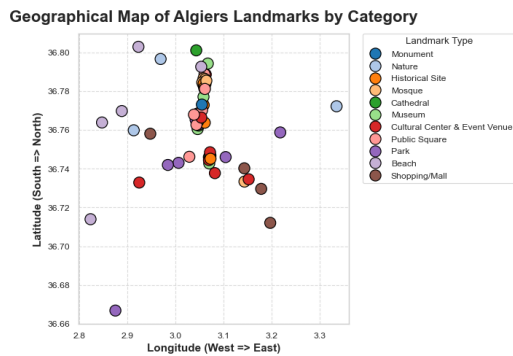
Type	Count	Type	Count	Type	Count
Cultural Center	11	Public Square	7	Shopping/Mall	4
Museum	9	Historical Site	6	Monument	2
Mosque	8	Nature	5	Cathedral	2
Park	7	Beach	5	Total	67

Table 3: Constraint Statistics

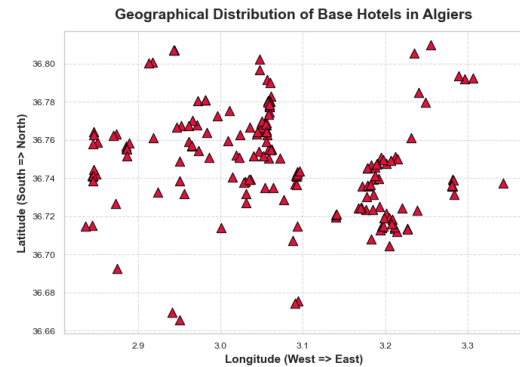
Metric	Avg	Min	Max
Rating	7.05/10	2.5/10	9.7/10
visiting time	61	10	180



(a) Distribution of Rating and estimated visiting time per landmark



(b) Geographical representation for landmarks



(c) Geographical representation for hotels

Figure 1: Exploratory Data Analysis visual representations.

5 Problem Solving Techniques

5.1 Problem Formulation

The problem is formulated as a constrained optimisation task applied to the city of Algiers, where the objective is to construct a personalised daily itinerary that maximises total cultural interest within a user-defined time budget. Given a fixed departure hotel, a set of candidate landmarks each characterised by an interest score, a visit duration, and time-dependent opening hours, the system must select and order a subset of landmarks that best rewards the tourist's experience while remaining logistically feasible. This formulation naturally captures the real-world tourist planning scenario: a visitor has limited time and seeks to maximise the quality of their experience, not simply minimise travel time.

Search Problem Elements:

- **Initial State:** The tourist departs from the hotel at a fixed start time, with an empty itinerary and the full time budget available.
- **State Representation:** A state is an ordered list of Landmark objects, where the sequence determines the arrival time at each stop and therefore drives constraint checking. This representation was chosen for its directness and compatibility with all algorithmic paradigms used: it supports both constructive approaches (Greedy, ACS) and iterative modification strategies (SA, Hill Climbing, GA, ABC), as well as slot-based assignment (CSP).
- **Actions:** At each step, the agent may append a candidate landmark to the current itinerary, provided doing so does not violate any hard constraint.
- **Transition Model:** Appending a landmark updates the current time by adding the travel time from the last visited node and the landmark's visit duration, advancing the state forward in time.
- **Constraints:** Three categories of hard constraints must be satisfied by every valid solution: (1) the cumulative travel and visit time, including the return trip to the hotel, must not exceed the time budget; (2) each landmark must be open at its computed arrival time, with optional waiting permitted if the landmark opens later in the day; and (3) an optional landmark-type filter may restrict the candidate set. Invalid solutions are rejected during construction or repaired post-crossover rather than penalised, ensuring that all evaluated solutions are feasible.
- **Goal Test:** For constructive approaches (Greedy, ACS), a solution is complete when no further landmark can be feasibly added within the remaining budget, at which point the agent returns to the hotel. For iterative methods (SA, Hill Climbing, GA, ABC), termination is governed by algorithm-specific stopping criteria such as temperature threshold, stagnation counter, or maximum number of iterations/restarts. For CSP, the search terminates when a complete consistent assignment across all slots is found, or when the backtracking procedure exhausts all possibilities.
- **Objective:** Maximise the fitness function $F(S)$, defined formally in Section 5.2.1, which rewards visiting high-interest landmarks while penalising unnecessary travel time.

5.2 Objective Functions and Heuristics

5.2.1 Unified Evaluation (Fitness) Function

To establish a standardized metric for comparing the final itineraries generated by all algorithms, and to serve as the internal fitness function for our local search and evolutionary algorithms (Simulated Annealing, Hill Climbing, and Genetic Algorithms), we defined the following unified objective function:

$$F(S) = w \sum_{i \in S} I_i - \left(t_{(H,1)} + \sum_{i=1}^{n-1} t_{(i,i+1)} + t_{(n,H)} \right) \quad (1)$$

Where:

- S represents the proposed solution (the ordered path of landmarks).
- I_i is the interest score of landmark i .
- $t_{(i,j)}$ represents the estimated travel time in minutes between node i and node j .
- H represents the base hotel (which does not carry an interest score).
- w is a constant weight parameter used to scale the interest score against the travel time.

Intuition and Parameter Tuning:

Solvers maximize $F(S)$ to prioritize spending the time budget experiencing high-rated landmarks over commuting. The weight w balances the scale difference between ratings (out of 10) and travel times (minutes).

Through rigorous empirical testing with values ranging from 1 to 30, we determined that $w = 7$ yields the optimal balance. This translates to a strict, real-world algorithmic interpretation: *spending up to 7 minutes of travel time is mathematically justified if it results in gaining at least 1 full unit of interest score.*

5.2.2 Algorithm-Specific Heuristics

A. Informed Search Heuristic (Greedy Approach)

For deterministic, step-by-step path construction, this heuristic minimizes cost by favoring landmarks with high scores (I_n) and short travel times from the current location (c) and back to the hotel (H):

$$h(n) = \frac{t_{(c,n)} + t_{(n,H)}}{I_n} \quad (2)$$

B. Swarm Intelligence Transition Rule (ACS & Hybrid ACS-SA)

For our Ant Colony System variations, the probability of an ant moving from node i to node j relies on a combination of collective memory and greedy desirability. We defined the transition evaluation score as:

$$P_{(i,j)} = [\tau_{(i,j)}]^\alpha \times [\eta_{(i,j)}]^\beta \quad (3)$$

Where:

- $\tau_{(i,j)}$ is the **Pheromone Score**, representing the historical success of previous ants traversing this edge, updated iteratively via the ‘evaporate and reinforce()’ mechanism.
- $\eta_{(i,j)}$ is the **Heuristic Value**, defined as $\frac{I_j}{t_{(i,j)} + \epsilon}$ (where ϵ is a small constant to prevent division by zero).
- α and β are hyperparameters controlling the influence of the pheromone trail versus the heuristic desirability.

During the testing phase, we evaluated various combinations of α and β . We observed that prioritizing the heuristic over the pheromone trail (specifically combinations like $\alpha = 1, \beta = 2.5$ or $\alpha = 0.5, \beta = 2.5$) produced the most consistent, high-quality itineraries. This configuration ensures the swarm maintains a strong bias toward high-value, nearby landmarks while still benefiting from the collective memory of the colony.

6 Application Design

The application was designed as a production-ready intelligent tourism platform that transforms complex itinerary optimization into an intuitive and visually engaging user experience. Inspired by the identity of Algiers, the system combines a user-centered frontend, a scalable FastAPI backend, and a structured PostgreSQL database to provide practical, responsive, and efficient tourist itinerary management.

From the beginning of the development process, the application was approached not as a simple academic prototype, but as a deployable real-world tourism service. Several design principles guided the implementation process:

- **Production-Ready and Scalable Architecture:** The platform was designed with a modular structure separating frontend and backend services, ensuring scalability, maintainability, and future extensibility.
- **User-Centered Experience:** Special attention was given to simplicity, intuitive navigation, and smooth interaction flow, allowing tourists to move naturally from discovering the city to generating personalized itineraries.
- **Cultural Identity and Immersive Design:** The application reflects the atmosphere of Algiers through visuals inspired by its Mediterranean architecture, coastline, and urban colors, while integrating realistic trip-planning interactions and map-based itinerary visualization.

6.1 Frontend and UI/UX Design Approach

The frontend application was developed using **React** combined with **Tailwind CSS**, enabling the creation of a modern, responsive, and highly interactive user interface. React allowed the platform to be structured into reusable components, simplifying state management and improving maintainability, while Tailwind CSS provided flexible utility-based styling for rapid UI development and consistent visual design across the application.

The UI/UX design aimed to create an immersive connection between the user and the city of Algiers through a visual identity inspired by the city's white architecture and Mediterranean blue tones.

The interface adopts a modern minimalist style with clean layouts, responsive design, smooth transitions, and carefully structured typography to ensure clarity, readability, and an engaging user experience across devices.

The user journey was intentionally divided into two major phases:

1. **City Discovery Phase:** The user explores the platform, learns about Algiers, views landmark categories, and gains contextual understanding of the city's historical and cultural importance.
2. **Action and Planning Phase:** The user specifies trip parameters such as departure hotel, available time budget, preferred landmark categories, and optimization constraints. The system then generates a personalized itinerary optimized by the selected AI algorithm.

One of the most important interface elements is the interactive map visualization. Instead of presenting itinerary results as plain text, the application displays the optimized route directly on a map interface, allowing users to clearly understand movement flow, landmark order, and estimated travel paths throughout the city. This significantly improves itinerary interpretability and practical usability.

The mapping system relies on **OpenStreetMap (OSM)**, a collaborative open-source geographic database project that provides free and editable world map data. The choice of OpenStreetMap was motivated by its flexibility, accessibility, and compatibility with custom routing and visualization systems. Since OSM data is freely licensed, it allows seamless integration for search, navigation, route visualization, and customized map styling without depending on proprietary mapping providers. This made it particularly suitable for displaying optimized tourist itineraries and landmark paths throughout the city of Algiers.

6.2 Backend Architecture

The backend system was developed using **FastAPI**, a modern high-performance Python framework particularly suited for AI-powered systems and asynchronous operations. The choice of FastAPI was motivated by several technical advantages directly aligned with the needs of this project.

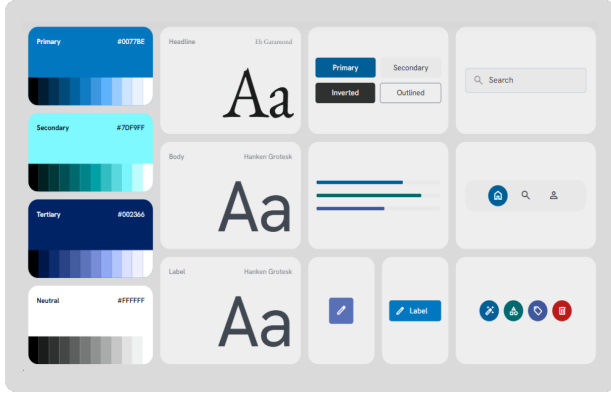
First, FastAPI provides excellent compatibility with Python-based optimization algorithms, enabling seamless integration of all implemented approaches including Simulated Annealing, Genetic Algorithms, ACS, ABC, CSP, and Hybrid methods. Since the majority of scientific computing and AI libraries are natively Python-oriented, FastAPI allowed rapid integration without introducing unnecessary interoperability overhead.

Second, the framework offers automatic API documentation generation through Swagger UI, simplifying testing, debugging, and future maintainability of the platform. The asynchronous request handling mechanism also improves scalability when handling simultaneous itinerary optimization requests from multiple users.

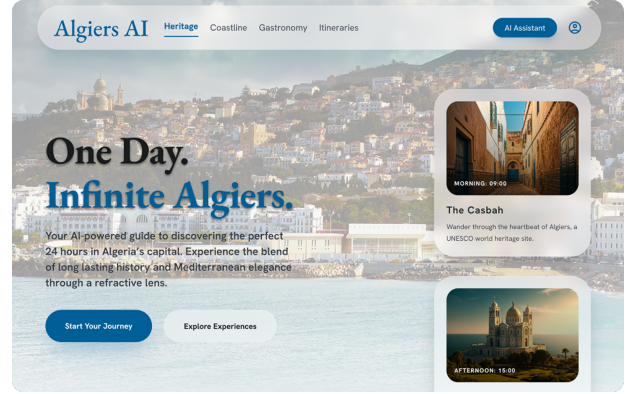
The backend architecture itself follows a modular service-oriented organization:

- **Data Management Layer:** Handles landmark, hotel, and route retrieval.
- **Optimization Engine Layer:** Executes the selected AI optimization algorithm and validates generated itineraries.
- **API Layer:** Exposes endpoints consumed by the frontend application.
- **Utility Services:** Manage geolocation processing, time conversions, filtering, and route formatting.

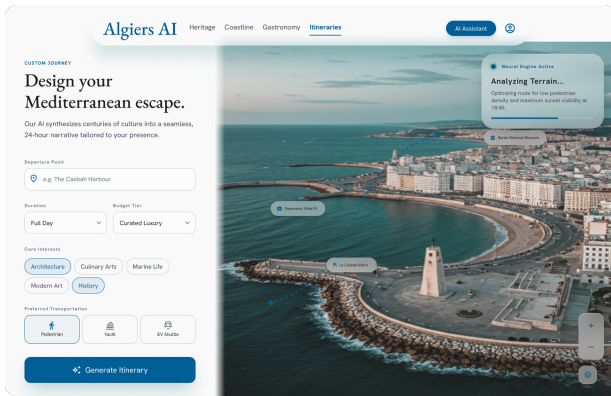
This separation ensures maintainability, easier debugging, and future extensibility of the platform.



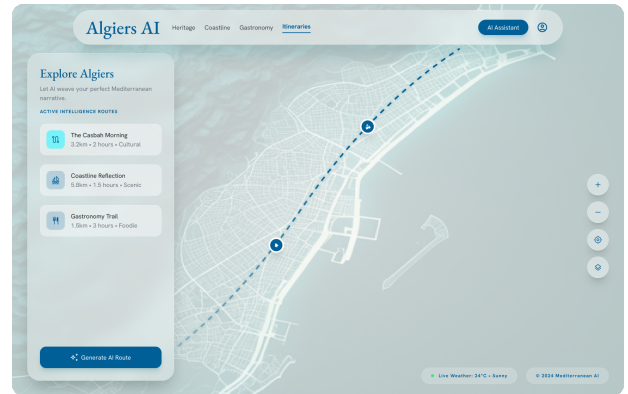
(a) Visual identity inspired by the colors and atmosphere of Algiers.



(b) Landing page mockup presenting the tourism platform experience.



(c) Interactive map visualization of the generated itinerary.



(d) Optimized path generation and landmark routing interface.

Figure 2: Application design and user interface overview.

6.3 Database Design

To support structured storage and efficient querying of tourism data, the application uses a **PostgreSQL** relational database system. PostgreSQL was selected due to its robustness, scalability, spatial querying capabilities, and strong compatibility with FastAPI-based architectures.

The database centralizes all project information including landmarks, hotels, opening hours, ratings, coordinates, categories, route matrices, and generated itineraries. The relational structure allows efficient indexing and filtering operations required during optimization and user query processing.

Additionally, PostgreSQL provides reliable consistency guarantees, which is particularly important for maintaining accurate geographical and scheduling information. The database schema was designed to remain extensible, allowing future integration of live traffic data, ticket pricing systems, user accounts, feedback systems, and dynamic recommendation modules.

Together, the combination of a responsive frontend experience, a scalable FastAPI backend, and a structured PostgreSQL database establishes a complete intelligent tourism platform capable of supporting real-world deployment scenarios while remaining highly adaptable for future AI-driven improvements.

7 Results and Analysis

7.1 Evaluation Setup

All experiments start from a fixed hotel at 9:00 AM across three time budgets (6 h, 12 h, 24 h). Each algorithm used its best parameter configuration, selected from individual parameter studies. Performance is measured by **Total Interest Score** ($7 \times \sum \text{ratings} - \sum \text{travel time}$) and **computation time**.

7.2 Results

Table 4: Total Interest Score (left) and Runtime in seconds (right) across time budgets

Algorithm	6h	12h	24h	6h	12h	24h
Greedy	392.2	939.8	1308.6	0.001	0.003	0.004
SA	632.2	1049.9	1515.0	34.04	96.29	179.76
GA	621.0	986.6	1264.0	0.57	0.96	1.39
ACS	627.5	1065.0	1412.2	6.60	12.01	14.87
ABC	616.2	1014.8	1398.4	99.00	210.71	384.32
Hill Climbing	639.2	1050.7	1507.5	25.36	67.91	176.91
CSP	272.9	529.5	1157.2	1.46	2.37	3.83
ACS-SA Hybrid	641.9	1045.1	1442.7	7.91	12.04	14.52

SA achieves the highest overall score (1 515.0 at 24 h), followed closely by Hill Climbing (1 507.5). CSP is the clear outlier, scoring roughly half the metaheuristics at every budget. Greedy is four to five orders of magnitude faster than any metaheuristic, but ranks second-lowest in quality.

7.3 Deep Dive — Main Algorithms

7.3.1 Greedy Best-First Search

Greedy scores 392.2 at 6 h — 61.2% below SA — visiting only 8 landmarks versus 12+ for every metaheuristic. Its myopic neighbour selection clusters routes around moderate-value nearby sites, systematically missing distant high-value landmarks such as the Great Mosque (9.7) and the Botanical Garden Hamma (9.6).

7.3.2 Simulated Annealing

SA outperforms Greedy by +61.2% at 6 h and +15.8% at 24 h. We used the famous exponential (geometric) cooling schedule to control the temperature, updated via $T_{new} = C \times T_{old}$. This schedule systematically drops the temperature quickly at first to avoid wasting time on random exploration, and then cools very slowly at the end to deeply exploit and fine-tune the best local solutions. The dominant parameter in this schedule is the cooling rate: $C = 0.99$ yields ≈ 745 while $C = 0.95$ drops to ≈ 715 (−4%); initial temperature has negligible impact ($< 0.5\%$). Figure 3 shows rapid improvement in the first 200 iterations, followed by deliberate acceptance of worse solutions to escape local optima. Three reheats push the best-so-far from $\approx 1\,010$ to $\approx 1\,040$, confirming reheating is essential under opening-hour constraints.

7.3.3 Genetic Algorithm

GA scores 6.0% below SA at 12 h yet runs $100\times$ faster (0.96 s vs 96.29 s). From a factorial study of 108 configurations, the optimal setup is rank selection, Order Crossover (OX1), and swap mutation at rate 0.10. OX1 preserves time-window feasibility across crossover. Performance plateaus at 125 generations with no further improvement.

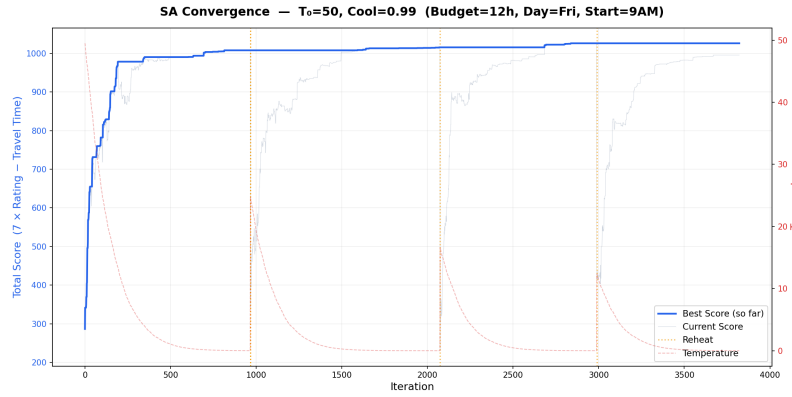


Figure 3: SA convergence ($T_0=50$, $C=0.99$, 12 h budget).

7.4 Additional Algorithms

7.4.1 Strong Performers

Hill Climbing matches SA within 0.5% at 24 h despite being fully deterministic. ACS achieves the highest 12 h score (1 065.0) in just 12 s. The ACS-SA Hybrid leads at 6 h (641.9) by combining global exploration with local refinement.

7.4.2 Weaker Performers

ABC is the slowest algorithm (384 s at 24 h) yet ranks only 5th in score. CSP visits as few as 5 landmarks at 6 h, functioning as a feasibility validator rather than an optimiser. Figure 4 summarises all eight algorithms at 12 h.

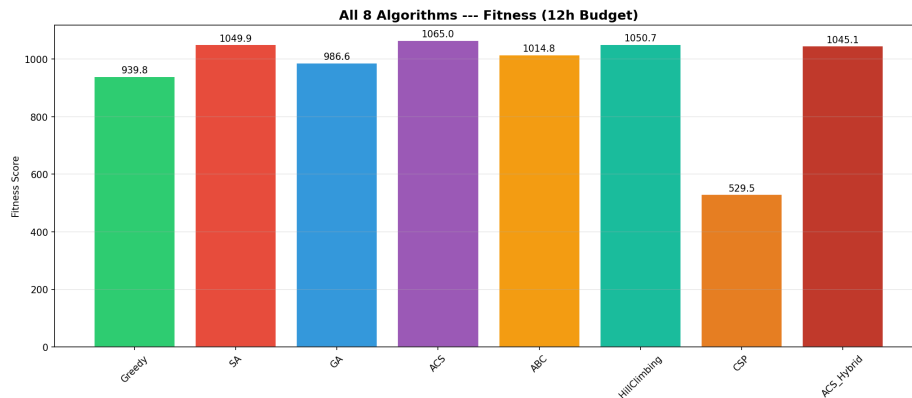


Figure 4: Total Interest Score for all eight algorithms at 12 h.

7.5 Time Budget and Runtime Analysis

Tightening the budget from 24 h to 6 h penalises CSP most severely (−76.4%) and Greedy (−70.0%), while metaheuristics lose 55–58%. GA is the most robust (−50.9%), maintaining population diversity even as the feasible region shrinks. At 6 h all metaheuristics cluster tightly (616–642), clearly separating from Greedy (392) and CSP (273). Regarding runtime, ABC is the extreme outlier (384 s at 24 h) despite producing only mid-tier scores. ACS offers the most compelling tradeoff: highest 12 h score (1 065.0) in 12 s — 8× faster than SA for a difference of only 1.4%. Figures 5a and 5b illustrate these trends.

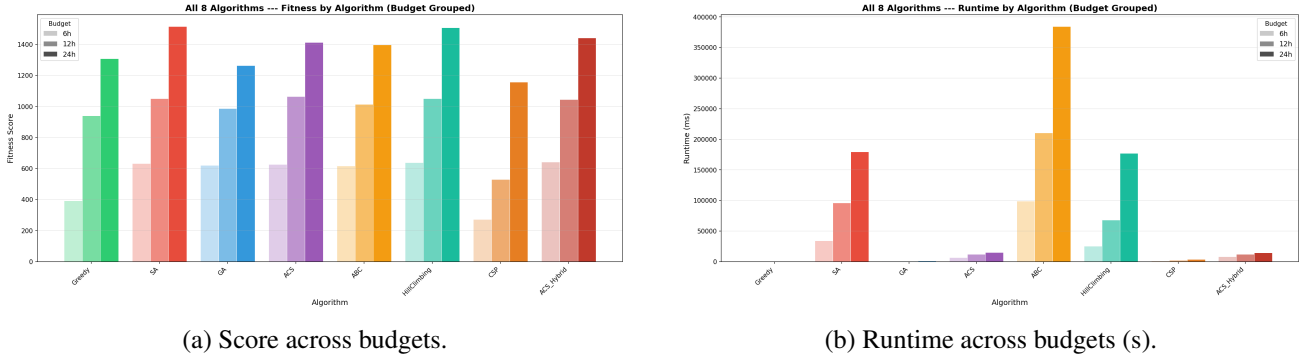


Figure 5: Time budget impact (left) and computation time (right) for all eight algorithms.

7.6 Key Insights

- **Best overall:** SA leads at 24 h (1 515.0); Hill Climbing is within 0.5%.
- **Most robust under tight budget:** GA degrades least at 6 h (−50.9%).
- **Best score/runtime tradeoff:** ACS — top 12 h score in 12 s, 8× faster than SA.
- **Most surprising:** Deterministic Hill Climbing matches SA at 24 h, outperforming all population-based methods.
- **Biggest disappointment:** ABC — slowest algorithm, yet only 5th in score.

8 Discussion

The experimental results across eight algorithms consistently reveal a core tension between solution quality, computation time, and constraint reliability — a trade-off that shapes every practical design decision in AI-driven tourist itinerary planning.

8.1 Results Interpretation

Across all budget levels, the Hybrid ACS-SA and Hill Climbing consistently delivered the highest solution quality, with HC reaching a peak fitness of 1555.0 at the 24h budget, while the hybrid approach offered the best quality-to-runtime trade-off overall. SA and ACS performed strongly and reliably, whereas GA stood out as the fastest high-quality solver, making it well-suited for time-sensitive scenarios. ABC proved computationally prohibitive relative to its gains, and CSP underperformed across all budgets due to its lack of an inherent optimisation mechanism. Collectively, the results confirm that metaheuristic and hybrid approaches significantly outperform both exact and greedy methods on this problem, with algorithm selection ultimately depending on the acceptable trade-off between solution quality and computation time.

8.2 Limitations

- **Static Problem Instance.** The experiments were conducted on a single problem instance with fixed landmarks, hotels, time matrix, and starting conditions. While this allows controlled comparisons between algorithms, it does not guarantee that performance rankings generalise to other cities, different landmark sets, alternative time budgets, or varying user preferences.
- **Fixed Hyperparameters.** Each algorithm was evaluated with a single set of hyperparameters (e.g., $ABC's_colony_ratio = 0.5$, $limit = 20$, $GA's_mutation_rate = 0.1$, $population_size = 100$, $ACS's_alpha = 1.0$, $beta = 2.5$, $rho = 0.1$). While these parameters were selected based on empirical tuning trials conducted during development, a more exhaustive systematic hyperparameter optimisation might improve performance for some methods.

- **No Optimality Baseline.** The absence of an exact solver or provably optimal solution for the tested instances makes it impossible to quantify the true optimality gap of any algorithm, limiting the strength of conclusions about absolute solution quality.

8.3 Potential improvements

- **Real-Time Constraint Handling.** The current implementation assumes perfect knowledge of travel times and opening hours. A more robust system would incorporate real-time data feeds for traffic conditions, unexpected attraction closures, and queue waiting times, then dynamically re-optimize the remaining itinerary.
- **Learning from User Feedback.** Incorporating a learning component that refines recommendations based on explicit user ratings or implicit behavior (e.g., which landmarks the user actually visited, how much time they spent) could personalize future itineraries.

8.4 Future Work

- **Systematic Hyperparameter Optimisation.** A rigorous hyperparameter study should be conducted for each algorithm. This would enable fairer cross-algorithm comparisons and might reveal that certain algorithms currently underperforming (e.g., ABC) could achieve competitive results with better parameters.
- **Integration with Real-World Travel Data.** Connecting the system to live APIs (Google Maps Distance Matrix for travel times, official attraction websites for opening hours and ticket prices) would enable practical deployment. This would also introduce additional challenges, including handling missing data, dealing with API rate limits, and maintaining data freshness.

9 Conclusion

This paper presented a comprehensive AI-driven solution to the Tourist Trip Design Problem applied to the city of Algiers. The system was designed to construct personalised, constraint-compliant daily itineraries by maximising total cultural interest within a user-defined time budget, subject to landmark opening hours and mandatory visit durations.

- **Key Findings.** This project demonstrated that hybrid and metaheuristic algorithms significantly outperform greedy and constraint-based methods for tourist itinerary optimisation. The Hybrid ACS-SA and Hill Climbing produced the highest quality solutions, while GA and ACS offered the best balance between solution quality and computational efficiency. SA delivered consistently strong results across all budget levels, and ABC, despite its exploratory nature, failed to justify its high runtime cost relative to competing approaches.
- **Contributions.** The primary contribution of this work is a modular, multi-paradigm optimisation engine applied to a curated Algiers landmark dataset, integrating eight algorithmic approaches under a unified evaluation framework and exposing the results through a FastAPI backend ready for deployment as a tourist-facing web service. The structured hyperparameter studies conducted for each algorithm, and the three-budget comparative methodology covering quality, runtime, and visit count, provide a reusable benchmarking framework applicable to itinerary planning in other cities and domains.
- **Implications.** The findings offer actionable guidance for real-world system design: applications requiring real-time responsiveness should favour GA or ACS, while offline planning scenarios where quality is paramount are better served by SA or Hybrid ACS-SA. More broadly, the results reinforce that algorithm selection in combinatorial optimisation is inherently context-dependent, and that hybridisation of complementary search strategies — as demonstrated by the ACS-SA pairing — is a consistently effective design principle worth pursuing in future work.

10 References

1. Chen, Y.-H., Sun, W.-J., & Chiang, T.-C. *Multiobjective Orienteering Problem with Time Windows: An Ant Colony Optimization Algorithm*.
2. Gao, S., Ruan, Z., Floor, P., Roveri, M., Palopoli, L., & Fontanelli, D. *Learning to Solve Orienteering Problem with Time Windows and Variable Profits*.
3. Granda, B., & Vitoriano, B. *The Team Orienteering Problem with Variable Time Windows*.
4. Işık, E. E., Ayyıldız, E., & Taşkın, A. *A Constraint Programming Approach for Multi-objective Tourist Trip Design Problem with Mandatory Visits: A Case Study for İzmir, Turkey*.
5. Larrañaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. *Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators*. Department of Computer Science and Artificial Intelligence, University of the Basque Country.
6. Dorigo, M., & Gambardella, L. M. (1997). *Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem*. Available at: https://iridia.ulb.ac.be/~mdorigo/Published_papers/All_Dorigo_papers/DorGam1997tec.pdf
7. *Generating Travel Itinerary Using Ant Colony Optimization*. Available at: https://www.academia.edu/58712906/Generating_Travel_Itinerary_Using_Ant_Collony_Optimization
8. *ACOSA / Clustering Hybrids for Optimization Enhancement*. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10557949/>
9. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). *Optimization by Simulated Annealing*. ScienceDirect Overview. Available at: <https://www.sciencedirect.com/topics/social-sciences/simulated-annealing>
10. *Reheat Cycles in Job Shop Scheduling*. Applied Soft Computing. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S1568494619305423>
11. *Enhanced Simulated Annealing for Tourist Recommendation Systems*. IEEE Xplore. Available at: <https://ieeexplore.ieee.org/document/6859737>
12. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson Education Limited.

11 Appendix A

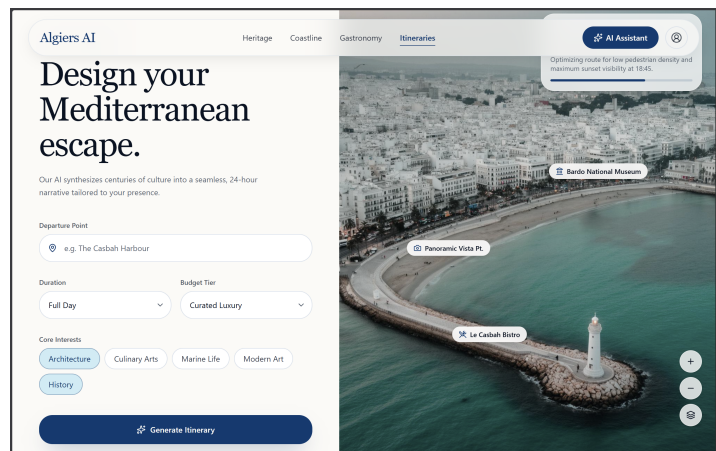


Figure 6: Landing page where users configure itinerary preferences, transportation mode, and trip constraints.

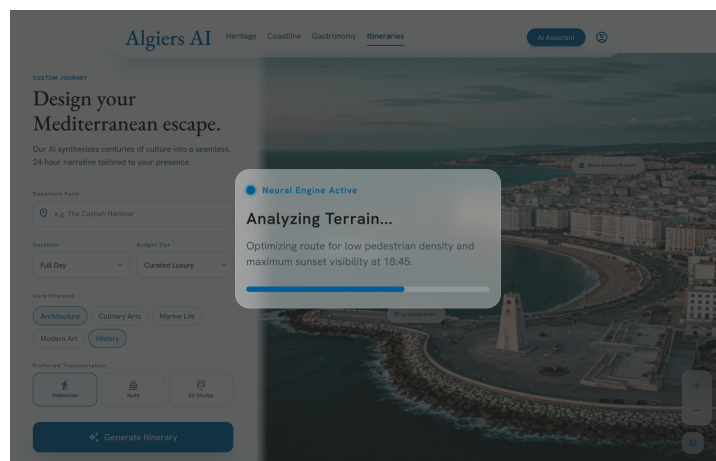


Figure 7: AI-powered itinerary generation process showing real-time route optimization and analysis.

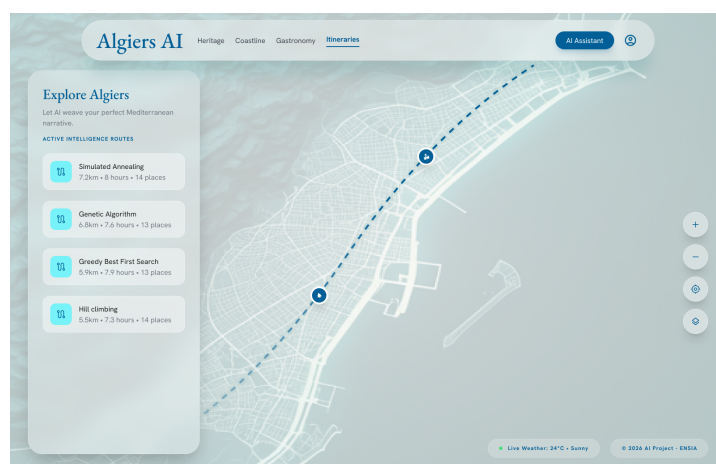


Figure 8: Interactive map interface displaying optimized tourist paths and landmark navigation across Algiers.

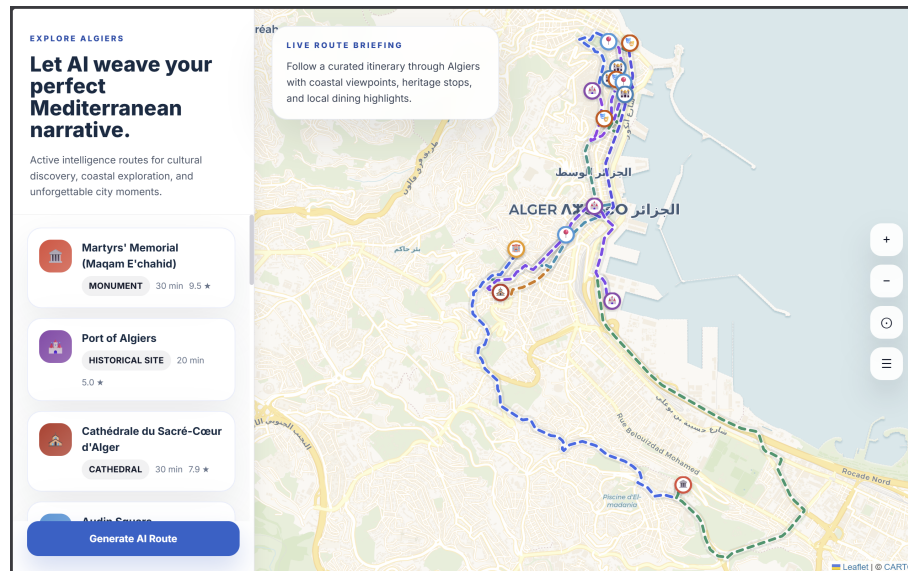


Figure 9: Detailed travel itinerary timeline UI showcasing curated stops in Algiers with immersive destination cards, AI travel insights, live route tracking, and journey optimization metrics in a clean modern interface.

12 Appendix B: Team Contributions

1. Dataset Construction & Preprocessing

- **Landmarks Data Collection:** Web scraping, GPS validation, rating assignment, visit durations, and image sourcing were collaboratively handled by *Ayoub Ouazene*, *Mohamed Ayoub Benamrouche*, *Mohamed Riad Doukha*, *Mohammed Al Ghazali Meghabber*, and *Rayane Maarouf*.
- **Hotels Dataset & Travel Constraints:** Hotel extraction, data loading functions, and OSRM Time Matrix generation were executed by *Mehdi Benhamoud*.

2. Problem Formulation

- **State Representation & Environments:** Local search and Ant Colony environments by *Ayoub Ouazene*; Informed search states and heuristics by *Rayane Maarouf*; CSP formulation by *Mehdi Benhamoud*.
- **Unified Objective Function:** Originally proposed by *Ayoub Ouazene*. The mathematical balancing and final formula were achieved through iterative testing and collaborative refinement by the *entire team*.

3. Problem Solving & Algorithm Implementation

Note: "Testing & Plotting" encompasses rigorous hyperparameter tuning (e.g., cooling rates, generation limits) and convergence visualization for each algorithm.

- **Greedy Best-First Search:** Implemented by *Rayane Maarouf*; Tested by *Mehdi Benhamoud*.
- **Simulated Annealing (SA):** Implemented by *Mohamed Riad Doukha*; Tested & Plotted by *Ayoub Ouazene* and *Mohamed Riad Doukha*.
- **Genetic Algorithms (GA):** Core implementation (including all crossover/mutation variants) by *Mohammed Al Ghazali Meghabber*; Tested & Plotted by *Mohamed Ayoub Benamrouche*.
- **Hill Climbing:** Implemented, tested, and plotted jointly by *Mohamed Ayoub Benamrouche* and *Mohammed Al Ghazali Meghabber*.
- **Ant Colony System (ACS):** Implemented by *Ayoub Ouazene*; Tested & Plotted by *Mohamed Riad Doukha*.
- **Hybrid ACS-SA:** Proposed and implemented by *Mohamed Riad Doukha*; Tested & Plotted by *Mohamed Riad Doukha* and *Ayoub Ouazene*.
- **Constraint Satisfaction (CSP):** Implemented, tested, and plotted by *Mehdi Benhamoud*.
- **Artificial Bee Colony (ABC):** Implemented, tested, and plotted entirely by *Rayane Maarouf*.

4. Results and Analysis, Discussion, Conclusion

- **Evaluation & Synthesis:** The comparative analysis between algorithms, result visualization, deep-dive discussions, and the final project conclusion were completely authored by *Rayane Maarouf*.

5. Application Development (Web Interface)

- **Visual Identity and UI/UX Design:** *Mohamed Riad Doukha*.
- **Frontend Development:** Main functionality engineered by *Mohamed Ayoub Benamrouche*; Auxiliary pages developed by *Rayane Maarouf*, *Mohamed Riad Doukha*, and *Mohammed Al Ghazali Meghabber*.
- **Backend & AI Integration:** Server architecture and algorithm endpoints built by *Mehdi Benhamoud* and *Ayoub Ouazene*.

6. Documentation & Report Writing

- **Formatting & Structure:** LaTeX template creation, page division, and formatting by *Mohammed Al Ghazali Meghabber*.
- **General Drafting:** Every team member contributed equally to the core writing process, producing a minimum of two pages each for their respective domains.